

Prática avaliativa 3: Estrutura de dados

Gabriel de Araújo Costa Gomes
2022102872

Departamento de Ciência da Computação; Universidade Federal de Minas Gerais
Belo Horizonte, Minas Gerais, Brasil; 11 de novembro de 2024
gacg2004@ufmg.br

1 Introdução

O objetivo desse relatório é analisar a complexidade de algoritmos recursivos e não recursivos que servem ao mesmo propósito e comparar o custo computacional entre eles.

2 Metodologia

Primeiramente, foram criados algoritmos que calculassem a posição n da sequência de Fibonacci e Fatorial de forma iterativa e recursiva. Em seguida, para que fossem feitas as análises, foram utilizadas para as funções Fatorial o valor 20 e para as funções Fibonacci o valor 13.

Utilizando a biblioteca `time`, foram medidos os tempos de execução para todas as configurações e comparados com os tempos de utilização de recursos de usuário e sistema. Para os algoritmos recursivos foram utilizados também a biblioteca `gprof` e comparados os resultados com a biblioteca `time`. Além da medição do algoritmo padrão, para melhor análise dos algoritmos recursivos, foram calculados os tempos dos algoritmos com uma função adicional que calcula o $1000000 * \sin n$ para cada chamada de função.

3 Análise dos Algoritmos

3.1 Algoritmos Iterativos

Fatorial Iterativo

Como podemos ver na tabela 1, o algoritmo utiliza por pouquíssimo tempo os recursos de sistema (`sys`) e de usuário (`user`) em comparação com os de execução. Também conseguimos visualizar que o algoritmo necessita de pouco tempo computacional. Isso é resultado da construção do algoritmo, que possui apenas um laço de 2 até n . Ou seja, a complexidade de tempo do algoritmo pertence ao conjunto $O(n)$.

Métrica	Tempo
real	0m2,976s
user	0m0,003s
sys	0m0,002s

Tabela 1: Tempos de execução do algoritmo `fatorialIterativo`

Fibonacci Iterativo

A tabela 2 nos mostra algo bem similar ao comportamento do algoritmo do Fatorial Iterativo, pouco tempo de uso dos recursos de sistema (`sys`) e de usuário (`user`) em comparação com os de execução. Isso acontece pois o algoritmo de Fibonacci Iterativo também tem ordem de complexidade de tempo $O(n)$. O tempo de execução é significativamente menor por conta das métricas da avaliação, sendo Fatorial de 20 e Fibonacci de 13, entretanto o comportamento é o mesmo.

Métrica	Tempo
real	0m0,977s
user	0m0,007s
sys	0m0,001s

Tabela 2: Tempos de execução do algoritmo `fibonacciIterativo`

3.2 Algoritmos Recursivos

Para os algoritmos recursivos foram feitos testes utilizando a função $\text{Seno}(n)$ que calcula 1000000 o seno de n a cada chamada de função.

Fatorial Recursivo

O algoritmo de Fatorial Recursivo, quando analisamos sua utilização sem a função $\text{Seno}(n)$ dentro dele, possui comportamento quase idêntico ao algoritmo de Fatorial Iterativo, tanto em tempo de execução quanto em tempo de utilização de recursos. O uso dos recursos de sistema (**sys**) é um pouco mais elevado, pelo fato de ser um algoritmo recursivo. Isso acontece pois a complexidade de tempo do algoritmo Fatorial Recursivo também é $O(n)$.

Como a função $\text{Seno}(n)$ tem complexidade $O(1)$, a adição dela no algoritmo interfere pouco nos tempos de execução e de utilização de recursos.

Métrica	Com Seno	Sem Seno
real	0m7,009s	0m1,918s
user	0m3,742s	0m0,003s
sys	0m0,005s	0m0,004s

Tabela 3: Tempos de execução do algoritmo `fatorialRecursivo` com e sem o cálculo do seno

O uso da biblioteca `gprof` não mostrou muita variação no tempo de execução em nenhum dos casos do algoritmo, como podemos ver na tabela 4.

Biblioteca	Com Seno	Sem Seno
time	7,009s	1,918s
gprof	5,436s	2,09s

Tabela 4: Tempos de execução do algoritmo `fatorialRecursivo` com e sem o cálculo do seno, medidos com `time` e `gprof`

Fibonacci recursivo

Já algoritmo de Fibonacci recursivo se difere bastante do Fibonacci Iterativo, com tempos de execução bem maiores. Isso se deve ao fato do algoritmo ter a complexidade de tempo de ordem $O(2^n)$. Por esse motivo foi escolhido a métrica do cálculo da 13ª posição da sequência de Fibonacci, para valores muito maiores o tempo de execução seria exponencialmente maior.

Métrica	Com Seno	Sem Seno
real	0m9,090s	0m5,008s
user	0m7,209s	0m0,004s
sys	0m0,016s	0m0,004s

Tabela 5: Tempos de execução do algoritmo `fibonacciRecursivo` com e sem o cálculo do seno

Diferente das medições feitas no algoritmo de Fatorial Recursivo, o uso da biblioteca `gprof` mostrou uma medição com uma variação maior.

Biblioteca	Com Seno	Sem Seno
time	9,090s	5,008s
gprof	10,39s	7,53s

Tabela 6: Tempos de execução do algoritmo `fibonacciRecursivo` com e sem o cálculo do seno, medidos com `time` e `gprof`

4 Conclusão

Esses experimentos mostram a importância de saber quando utilizar algoritmos recursivos, pois como no exemplo do Fibonacci, que era uma boa solução de complexidade linear em tempo usando o algoritmo iterativo, se tornou uma péssima solução quando utilizamos a recursividade. Para valores tão pequenos quanto usados nos testes pode

até não ser tão significativo, mas pra valores maiores o algoritmo recursivo pode se tornar inutilizável, durando horas ou ate dias pra computar a solução.